

Pandas para Análise de Dados com Python

“Te ensino TUDO sobre Pandas para análise de dados com Python” — Juliano, Asimov Academy (~12 min)

O vídeo é uma passada rápida pelas operações essenciais do Pandas, que o autor descreve como o “Excel do Python”. O exemplo usado do começo ao fim é uma tabela de funcionários com nome, idade, salário, departamento e status de ativo.

1. Instalação e importação

```
pip install pandas          # já vem no Colab e no Anaconda
import pandas as pd
```

2. Estruturas principais

- **DataFrame**: a tabela, com linhas e colunas. Pode ser criado a partir de um dicionário.
- **Series**: uma única coluna.

3. Leitura de arquivos

```
df = pd.read_csv("funcionarios.csv")
df = pd.read_excel("arquivo.xlsx", sheet_name="Aba")
```

4. Navegação e seleção

```
df.head() / df.tail()      # 5 primeiras / últimas linhas
df[["nome", "idade"]]      # várias colunas → DataFrame
df["idade"]                # uma coluna → Series
```

5. Filtros

Uma comparação como `df["idade"] > 30` gera uma **série booleana** (True/False). Ao passá-la para dentro do DataFrame, ela vira um filtro. Para combinar condições, usa-se `&`:

```
f1 = df["idade"] > 30
f2 = df["salario"] > 5000
df[f1 & f2]
```

6. Criar e remover colunas

```
df["salario_anual"] = df["salario"] * 12
df = df.drop("salario_anual", axis=1)  # axis=1 indica coluna
```

7. Resumos

- `df.info()` mostra o tipo de cada coluna e se há nulos.
- `df.describe()` dá as estatísticas das colunas numéricas: média, desvio padrão, mínimo, mediana e máximo.
- Para uma coluna específica, há `.sum()`, `.mean()`, `.median()`, `.max()` e `.min()`.

8. Limpeza de dados (nulos e tipos)

```
df.dropna()                # remove linhas com qualquer nulo
df.dropna(subset=["idade"]) # só onde "idade" é nula
df["idade"] = df["idade"].fillna(df["idade"].mean()) # preenche com a média
df["idade"] = df["idade"].astype(int)              # float → int
```

9. Contagem e agrupamento

```
df["ativo"].value_counts() # ex.: 11 ativos, 9 inativos
df.groupby("id_departamento")["salario"].mean() # média por departamento
# também .sum(), .max() etc.
```

10. Ordenação

```
df.sort_values(by="salario") # crescente
df.sort_values(by="salario", ascending=False) # decrescente
```

11. Combinar tabelas

- **merge** funciona como um JOIN e traz o nome do departamento pela coluna em comum. Se os nomes das colunas forem diferentes, usa-se `left_on` / `right_on`.
- **concat** empilha tabelas com as mesmas colunas, uma embaixo da outra.

```
df.merge(df_dept) # ou left_on=..., right_on=...
pd.concat([df, df_novos], ignore_index=True)
```

12. Exportar

```
df.to_csv("todos_funcionarios.csv")
df.to_excel(...)
```

Equivalências com SQL

Pandas	SQL
<code>df[f1 & f2]</code>	WHERE ... AND ...
<code>df.groupby(...).mean()</code>	GROUP BY ... AVG()
<code>df.merge(...)</code>	JOIN
<code>pd.concat([...])</code>	UNION ALL
<code>df.sort_values(...)</code>	ORDER BY
<code>df["col"].value_counts()</code>	COUNT(*) ... GROUP BY col

Mensagem final do vídeo: o poder do Pandas aparece quando ele é combinado com o resto do Python para resolver problemas reais, como relatórios automáticos, dashboards e sistemas internos.